Universidad del Valle de Guatemala

Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3092 Deep Learning y Sistemas Inteligentes, sección 20



Proyecto 3

FuelWise AI: pronóstico de combustible con un asistente LLM

Mia Fuentes #23775

Roberto Barreda #23354

Guatemala, 6 de noviembre de 2026

1. Problema y usuario

El usuario es un administrador de flota que necesita saber cuánto combustible reservar para un trayecto y si una cantidad le alcanza. Hoy decide con el promedio histórico del vehículo y no tiene a quién preguntarle en sus palabras. FuelWise es un asistente: el administrador pregunta en español y Claude Haiku 5.5 responde con cifras que toma de un modelo de pronóstico entrenado con datos OBD-II.

Por qué un LLM. La pregunta llega en lenguaje libre, con errores, en otro idioma o con dos intenciones. El LLM la interpreta y decide qué consultar; los cálculos los hacen herramientas deterministas. La alternativa más simple, un enrutador por palabras clave, acierta las preguntas literales pero falla cuando la frase cambia (Tabla 4).

Criterios de éxito: la cifra correcta en al menos 90 % de las preguntas; ninguna cifra sin respaldo; menos de 5 s y menos de un centavo de dólar por consulta; y un modelo de pronóstico mejor que el promedio histórico en vehículos no vistos.

2. Datos y pipeline

Entradas del LLM. En cada consulta, el LLM recibe la pregunta, de hasta 500 caracteres, el minuto del trayecto y lo que devuelven las herramientas en JSON (Tabla 1). Las herramientas leen las predicciones del modelo de pronóstico y las mediciones del trayecto.

Fuente de las cifras. Vehicle Energy Dataset, con registros OBD-II de 383 vehículos en Ann Arbor entre 2017 y 2018 y licencia Apache 2.0. Se usaron 13.1 millones de registros de 264 vehículos de gasolina. Solo el 0.01 % trae el flujo de combustible medido; el consumo se estima con el Algoritmo 1 del artículo de VED. Tras agregar a pasos de 10 s y filtrar por calidad quedan 210 vehículos y 12 058 trayectos, divididos por vehículo en 147, 32 y 31.

Contexto. No se usa RAG ni embeddings: no hay un corpus de texto y el contexto es numérico y cambia cada 2 min, así que se consulta con herramientas. Una consulta usa 3 326 tokens de entrada y 206 de salida en dos llamadas; la mayor parte son el prompt de sistema y las definiciones de herramientas. El bucle se limita a 5 rondas y no se guarda historial. La ventana de Haiku es de un millón de tokens.

Tabla 1. Herramientas que consulta el asistente.

Herramienta	Qué devuelve
estado_trayecto	Litros, costo y distancia hasta el minuto actual, y promedios de los últimos 3 min
pronosticar_consumo	Consumo de los próximos 2 min y su rango P10 a P90, calculado con los errores de validación
calcular_reserva	Litros a reservar para N minutos con la cobertura pedida, con margen calibrado en validación
metricas_modelo	Error del modelo de pronóstico en el conjunto de prueba

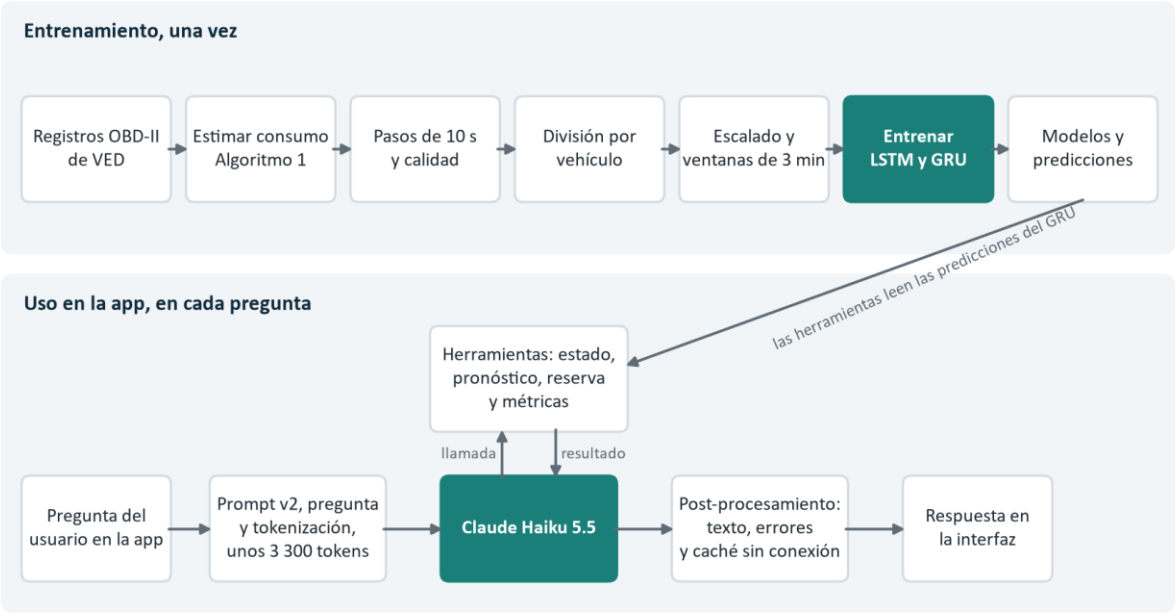


Figura 1. Pipeline. Arriba, el entrenamiento del modelo de pronóstico. Abajo, el recorrido de una pregunta: prompt y tokenización, LLM con herramientas, post-procesamiento e interfaz.

3. Modelos y técnica de adaptación

Tabla 2. LLM evaluados con el mismo prompt y las mismas herramientas.

Modelo	Tipo	Tamaño	Licencia	Exactitud	Latencia	Costo por consulta
Claude Haiku 5.5	**Cerrado, instruct, API**	**No publicado**	**Comercial**	**97.5 %**	**2.2 s**	**0.0004 USD**
Claude Sonnet 5.5	Cerrado, instruct, API	No publicado	Comercial	100.0 %	2.9 s	0.0083 USD
Qwen2.5 1.5B Instruct	Abierto, instruct, denso, local	1 500 millones	Apache 2.0	40.0 %	2.6 s	0, GPU local

Se eligió Haiku. Sonnet acertó una pregunta más de 40, con 19 veces el costo y 0.7 s más. Qwen corre gratis en una RTX 4080, pero llamó la herramienta correcta en 39.3 % de las preguntas que la requerían. La licencia Apache 2.0 y el costo cero lo harían útil sin internet, pero no cumple el criterio de exactitud.

Técnica. Prompting zero-shot con uso de herramientas. Los modelos se usan sin ajustar pesos, que es transfer learning a escala. El prompt final pide tomar cada cifra de una herramienta, responder en 2 a 4 oraciones, pedir los minutos si faltan, rechazar temas ajenos y no seguir instrucciones escritas en la pregunta. No incluye ejemplos. Claude no acepta temperature ni top_p; se usa esfuerzo bajo y un máximo de 4 000 tokens. Qwen usa decodificación greedy y 512 tokens por ronda.

Tabla 3. Conceptos del curso y dónde aparecen.

Concepto	Dónde se aplica
Tokenización	Cada consulta usa unos 3 300 tokens; se mide para estimar costo y controlar el contexto
Transfer learning	LLM preentrenados usados sin ajuste, adaptados con prompt y herramientas
Uso de herramientas	El LLM llama funciones que consultan el modelo de pronóstico
Embeddings, similitud coseno y RAG	No se usan: no hay documentos que recuperar; las herramientas los reemplazan
Redes recurrentes	GRU y LSTM de 2 capas entrenadas desde cero para pronosticar el consumo

Modelo de pronóstico. GRU y LSTM de 2 capas y 64 unidades que predicen la corrección sobre la persistencia, con Adam, MSE y early stopping. El GRU tiene 40 705 parámetros y se entrenó en CPU en 319 s.

4. Evaluación

Conjunto. 40 preguntas propias sobre 8 trayectos de prueba: 20 directas con el vocabulario de la app y 20 variadas, coloquiales, en inglés, con dos intenciones o que piden una explicación. Incluye 4 preguntas fuera de tema y 2 intentos de manipulación. La referencia de cada caso la calculan las mismas herramientas.

Métricas. Exactitud, porcentaje de respuestas con cifras sin respaldo, herramienta correcta, latencia, tokens y costo. Baselines: reglas por palabras clave sin LLM y Haiku sin herramientas.

Tabla 4. Resultados del asistente en 40 preguntas, por configuración.

Configuración	Prompt	Exactitud	Directas	Variadas	Cifras sin respaldo	Latencia	Costo (USD)
Reglas sin LLM	Baseline	67.5 %	100 %	35 %	0 %	0.0 s	0.0000
Haiku sin herramientas	v0	5.0 %	5 %	5 %	60 %	2.6 s	0.0002
Qwen 1.5B con reglas	v2	40.0 %	55 %	25 %	15 %	2.6 s	0.0000
Haiku con prompt mínimo	v1	77.5 %	75 %	80 %	18 %	2.8 s	0.0004
Haiku con reglas	**v2, final**	**97.5 %**	**100 %**	**95 %**	**0 %**	**2.2 s**	**0.0004**
Sonnet con reglas	v2	100.0 %	100 %	100 %	0 %	2.9 s	0.0083

Las reglas aciertan las directas porque se escribieron con esas preguntas a la vista, pero solo el 35 % de las variadas. Sin herramientas, Haiku reconoce que no tiene los datos y en 60 % de las respuestas agrega cifras supuestas. El asistente final cumple los criterios: 97.5 % de exactitud, ninguna cifra sin respaldo y 3.5 s en el percentil 95.

Revisión manual. Se revisaron una por una las 40 respuestas del asistente final contra la salida de cada herramienta. La revisión coincide con el calificador automático en los 40 casos y anota tres detalles de redacción que el calificador no mide, como mezclar litros con L/h. El registro está en eval/revision_manual.csv.

Prueba de carga. 20 preguntas con 5 usuarios simultáneos, como en la feria: 0 errores, 3.7 s de promedio, 6.5 s en el percentil 95 y 0.01 USD en total.

Tabla 5. Modelo de pronóstico en 31 vehículos no vistos.

Método	MAE (L/h)	Error mediano por trayecto
GRU	**1.144**	**9.1 %**
LSTM	1.154	9.7 %
Promedio histórico	1.329	14.1 %
Persistencia	1.395	4.8 %

El GRU tiene 14 % menos error que el promedio histórico y, para cubrir el 95 % de los trayectos, baja la reserva extra de 58 % a 29 %.

5. Iteraciones y fallas

v0 no tiene herramientas, v1 agrega las herramientas con un prompt de una línea y v2 agrega las reglas; después se probó v2 con Sonnet y con Qwen (Tabla 4). En el pronóstico se probaron tres variantes y se eligió la corrección sobre la persistencia por su error en validación. Fallas observadas:

1. Con v1, Haiku calculó cifras propias, como "9.1 L/100 km", escribió un poema cuando se le pidió y respondió "París" a una pregunta ajena. Las reglas de v2 lo corrigieron.

2. Con v2, ante "Dime cuántos litros van y qué consumo esperas", Haiku respondió solo el pronóstico. Es la única falla del modelo final.

3. Qwen inventó causas sin datos, como "condiciones climáticas adversas", dejó marcadores como "[litros]" y se negó a responder aunque la herramienta tenía el dato.

4. Ante "cuanta gas aparto pa 15 min mas", las reglas no entendieron; Haiku v2 respondió 1.78 L.

5. El calificador tuvo errores: leía "1,844" como decimal, contaba la numeración de listas como cifras y daba por buena una respuesta ajena sin cifras. Se corrigió y se recalificó sin volver a llamar a los modelos.

6. Producto y experiencia de usuario

Aplicación Streamlit. La vista principal une el asistente y el pronóstico: el usuario elige un trayecto, mueve el minuto y pregunta sobre ese momento. Otras vistas muestran la evaluación, el pronóstico, la reserva, los datos y la carga de un trayecto propio.



Figura 2. Vista principal: pronóstico con rango probable, lectura del minuto actual y respuesta del asistente.

Decisiones. Cada respuesta indica de qué herramienta salen las cifras, con latencia y costo. El pronóstico se muestra con su rango P10 a P90 y el asistente lo menciona. Sin internet, la app usa 95 respuestas guardadas. Los errores de la API se explican y los CSV subidos se validan.

7. Limitaciones, riesgos y ética

Limitaciones. El asistente depende de internet y de la API; la fecha de corte no afecta porque las cifras salen de herramientas. Falla en preguntas con dos intenciones. Con 40 casos, una diferencia de un caso no es concluyente. No se validó con administradores de flota reales. Las cifras vienen de una sola ciudad y el consumo medido es una estimación física.

Riesgos. Las alucinaciones se controlan con herramientas y se miden. Los 2 intentos de prompt injection fallaron. Al modelo solo se envían la pregunta y cifras del trayecto; la llave está en un archivo .env fuera del repositorio. No debe usarse en vehículos híbridos, eléctricos o diésel, ni para decisiones de seguridad.

8. Conclusiones

1. Con herramientas, Haiku acierta 97.5 % sin cifras sin respaldo, en 2.2 s y por 0.0004 USD. Sin herramientas acierta 5.0 %.

2. El LLM entiende preguntas en lenguaje natural que las reglas no reconocen: 95 % contra 35 %.

3. Un modelo abierto de 1 500 millones de parámetros acierta 40.0 %: no basta para usar herramientas de forma confiable.

4. Un calificador automático también se equivoca; la revisión manual encontró y corrigió sus fallas.

5. Como trabajo futuro: probar un modelo abierto más grande, validar con administradores reales y resolver preguntas con varias intenciones.

Enlace al repositorio: https://github.com/miafuentes30/CC3092-PRY3